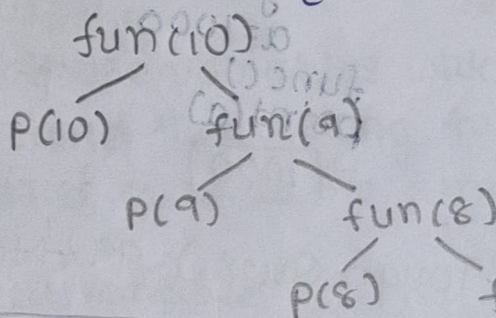


★ Recursion (Very Imp)

⇒ Whenever a function calls itself, it is called recursion and the function is called recursive function.

```
def fun(n):
    print(n)
    fun(n-1)
fun(10)
```



∴ We should always write base condition

```
def fun(n):
```

```
    if n > 1:
```

```
        print(n)
```

```
        fun(n-1)
```

```
fun(5)
```

O/p ⇒ 5
4
3
2

```
def fun(n):
```

```
    if n > 1:
```

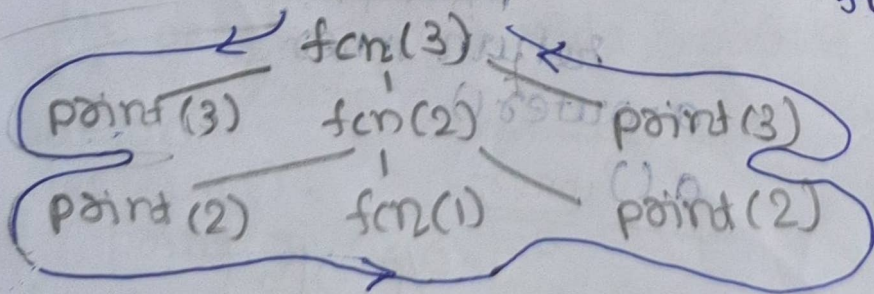
```
        print(n)
```

```
        fun(n-1)
```

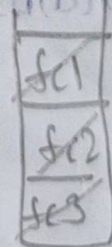
```
        print(n)
```

fun(3)

(Always Make Tree for better understanding)



⇒ 3 2 2 3 O/p




```

def fun(n):
    if n==1:
        return 1
    return n+fun(n-1)
print (fun(5))

```

15

```

def fun(n):
    if n==1:
        return 1
    return n*fun(n-1)
print (fun(5))

```

120

⇒ 15

120
fun(5)

5 * fun(4)

4 * fun(3)

3 * fun(2)

2 * fun(1)

★ Types of Recursion

i) Finite Recursion v/s Infinite Recursion

ii) Head Recursion v/s Tail Recursion

```
def fun(n):
```

```
    if n>1:
```

```
        fun(n-1)
```

```
        print(n)
```

Head

```
def fun(n):
```

```
    if n>1:
```

```
        print(n)
```

```
        fun(n-1)
```

Tail

iii) Direct Recursion v/s Indirect Recursion

A() → A()

A() → B()
B() → A()

que def A(n):
 if n>1:
 print(n)
 B(n-1)

```
def B(n):
    if n>1:
        print(n)
        A(n-1)
```

A(5)

A(5) → print(5)

B(4) → print(4)

A(3) → print(3)

B(2) → print(2)

X

O/P = 5 4 3 2

Finite / Infinite
Head / Tail
Direct / Indirect
Nested Recursion


```

def fun(n):
    if n <= 1:
        return 1
    return fun(n-1) + fun(n-2)
print(fun(5))

```

⇒ 8

Que (Nested fn)

```

def outer():
    a = 10
    def inner():
        a = a + 7 → Error
    inner()
    print(a)
outer()

```

⑧ fun(5) → 8
 ⑤ fun(4) + fun(3) → 3
 ③ fun(3) + fun(2) → 2
 ② fun(2) + fun(1) → 1
 fun(1) + fun(0) → 1

[You cannot modify the value of outer fn in inner fn]

[You cannot write global keyword]

[Use "non local a" keyword to modify Outer fun Variable]

Problem Solving

★ Nested Recursion

⇒ When a recursive call is passed as a parameter in recursive functions.

```

def fun(n):
    if n <= 1:
        return 1
    return fun(fun(n-1))

```

★ Problem Solving

Que

```
def NR(n):
    if n > 100:
        return n-10
    return NR(NR(n+11))
NR(95)
```

⇒ 91

Que

```
def func-A(n):
    if n > 0:
        print(n)
        func-B(n//5)
```

```
def func-B(n):
    if n > 0:
        print(n)
        func-A(n-1)
```

func-A(20)

func-A(20)

↓

func-B(4)

↓

func-A(3)

↓

func-B(0)

p(20)

(n//5): (20//5) = 4

p(4)

p(3)

(n//5): (3//5) = 0

0 > 0 ×

⇒ 20
4
3

95
+11
106

NR(95)

NR(NR(106))

96 (14)

NR(NR(107))

97 (14)

NR(NR(108))

98 (14)

98
+11
109

NR(NR(109))

99 (14)

NR(NR(110))

100 (11)

NR(NR(111))

101

• 106 > 100 ✓
↳ n-10 ⇒ 96
• 96 > 100 × ⇒ NR(NR(96))
• 107 > 100 ✓
↳ n-10 ⇒ 97
• 97 > 100 × ⇒ NR(NR(97))
• 108 > 100 ✓
↳ n-10 ⇒ 98
• 98 > 100 × ⇒ NR(NR(98))
• 109 > 100 ✓
↳ n-10 ⇒ 99
• 99 > 100 × ⇒ NR(NR(99))
• 110 > 100 ✓
↳ n-10 ⇒ 100
• 100 > 100 ⇒ NR(NR(100))
• 101 > 100 ✓
↳ n-10 ⇒ 101
• 101 > 100 ✓
↳ 101-n ⇒ 91

Que

```
def foo(n):
```

```
    if n == 0:
```

```
        return 0
```

```
    if n == 1:
```

```
        return 1
```

```
    if n == 2:
```

```
        return 3
```

```
    return n + foo(n-1) + foo(n-2)
```

foo(6):

(46) [6 + foo(5) + foo(4)]

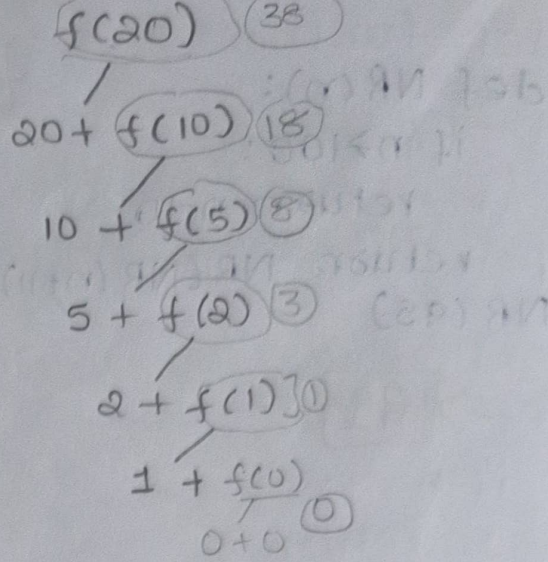
26 [5 + foo(4) + foo(3)]

14 [4 + foo(3) + foo(2)]

7 [3 + foo(2) + foo(1)]

⇒ 46

Que def f(n):
 if n==0:
 return 0
 result = n + f(n//2)
 return result
 c=f(20)
 print(c) $\Rightarrow$ 38



Que def count(child-dict, i):
 if i not in child-dict.keys():
 return 1
 ans = 1
 for j in child-dict[i]:
 ans += count(child-dict, j)
 return ans

0, not in keys X
 ans = 1
 # [1, 2] → [0] [3, 4, 5] → [1]
 ans = ans + count(child-dict, j)
 ans = 1 + count(child-dict, 1)
 + count(child-dict, 2)

create dict
 child-dict = dict()
 child-dict[0] = [1, 2]
 child-dict[1] = [3, 4, 5]
 child-dict[2] = [6, 7, 8]
 print(count(child-dict, 0))

$\Rightarrow$ 9

child-dict = {0: [1, 2], 1: [3, 4, 5], 2: [6, 7, 8]}

count(child-dict, 0) 9

1 + count(child-dict, 1) + count(child-dict, 2) 4 4

1 + count(child-dict, 3) + count(child-dict, 4) + count(child-dict, 5) 1 1 1

1
 3 is not in key 4

1 + count(child-dict, 6) +
 1 + count(child-dict, 7) +
 count(child-dict, 8) 1 4

def count(n, jumps):

if n == 0:

return 1

total = 0

for j in jumps:

if n - j >= 0:

total += count(n - j, jumps)

return total.

print(count(4, [1, 3, 5]))

count(4, [1, 3, 5])

count(3, [1, 3, 5]) + count(1, [1, 3, 5])

count(2, [1, 3, 5]) + count(0, [1, 3, 5])

count(1, [1, 3, 5])

count(0, [1, 3, 5])

return 1 (n == 0)

def f(n)

if n == 0:

return 0

return f(n-1) + g(n-1)

def g(n):

if n == 0:

return 1

return g(n-1) + f(n)

g(g(2))

=> 89

t=1

n=4

j=1

j=3

j=5

4-1=3

4-3=1

4-5=-1

n=3

j=1

j=3

j=5

3-1=2

3-3=0

3-5=-2

n=2

j=1

j=3

j=5

2-1=1

2-3=-1

2-5=-3

n=1

j=1

j=3

j=5

1-1=0

1-3=-2

1-5=-4

=> 3

g(5)

g(4) + f(5)

f(4) + g(4)

g(3) + f(4)

f(3) + g(3)

g(2) + f(3)

f(2) + g(2)

g(1) + f(2)

f(1) + g(1)

g(0) + f(1)

f(0) + g(0)

g(0) + f(0)

f(0) + g(0)

g(0) + f(0)

f(0) + g(0)

g(0) + f(0)

f(0) + g(0)

g(0) + f(0)